

REPORT: Project 1

CONVOLUTION

1. How would you describe the convolution process to another student?

First, you should make sure you have imported the libraries you will need to do manual convolution. The two most important ones I used were cv2 and NumPy. Then, you should initialize a kernel, as well as picking your padding value (or values) and stride size. It's also important to initialize a variable that holds your image and one that transforms the image into a grayscale format. Once you have all of your parameters set up, you can begin the convolution subroutine.

There are five main steps to manual convolution, the first is flipping the kernel. What this means is you will reverse the rows and reverse the columns, essentially rotating the values 180 degrees. What starts on the top left will end up on the bottom right and vice versa. We do this because we want the output of the convolution to depend only on the current and past inputs. Flipping the kernel is most important with asymmetric kernels to ensure a correct output, but it's still necessary for symmetrical ones as well.

The second step in manual convolution is adding padding to the input image. To pad an image you first have to decide what type of padding you want, this choice will have impacts on the edges of the output and will determine if you lose any of the image or if it blurs on the sides. In my convolution subroutine I implemented "same" padding, which bases the number of pixels you add on the edge of your image on the size of the kernel you have chosen. This style of padding ensures that all the pixels in the original image influence the convolution process.

The third step is to initialize an output matrix full of zeros that will hold the values that result from the elementwise multiplication of the image array and the kernel. This step is important because you are setting aside memory ahead of time, which speeds up the computation process in the next step. Otherwise, the computer would have to constantly be resizing the block of memory it is storing the output of the elementwise multiplication in.

The bulk of the computation involved in manual convolution is done in the fourth step. In this step you move the flipped kernel across the padded image one pixel (or more depending on your stride size) at a time. For example, if your stride was 2 you would move 2 pixels at a time which causes your level of detail to diminish because you skip certain pixels. At each position, the kernel is centered over a region of the image that matches its size (e.g., a 3×3 kernel covers a 3×3 pixel region). This is usually done with a set of nested "for" loops, which allows you to move horizontally and vertically. For each

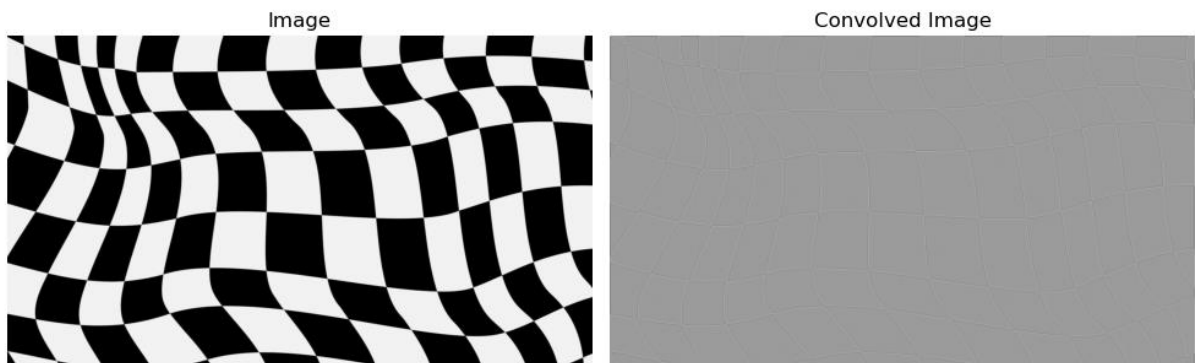
region you pass, we multiply each element of the region with the corresponding element of the flipped kernel. Then we sum all the products and this becomes the new pixel value which we store in the output matrix (i.e. the “result”). Mathematically, we are viewing the input region and the flipped kernel as vectors and then doing the dot product, a function which allows you to view the similarity between two vectors. When every pixel has undergone this transformation, the convolution is completed, and we do the final step, returning the output matrix.

In order to display the convolved image, we often need two more steps. First, it can be helpful to normalize the output matrix so that the results fall between 0 and 255. We do this because image displays typically have predefined ranges, and in the case of 8-bit images 0 represents black and 255 represents white. Second, you have to use some sort of code to actually display the data because in essence the output matrix is just a 2D array of values. This can be done with code from libraries like Matplotlib, which includes plenty of commands for visualization of data.

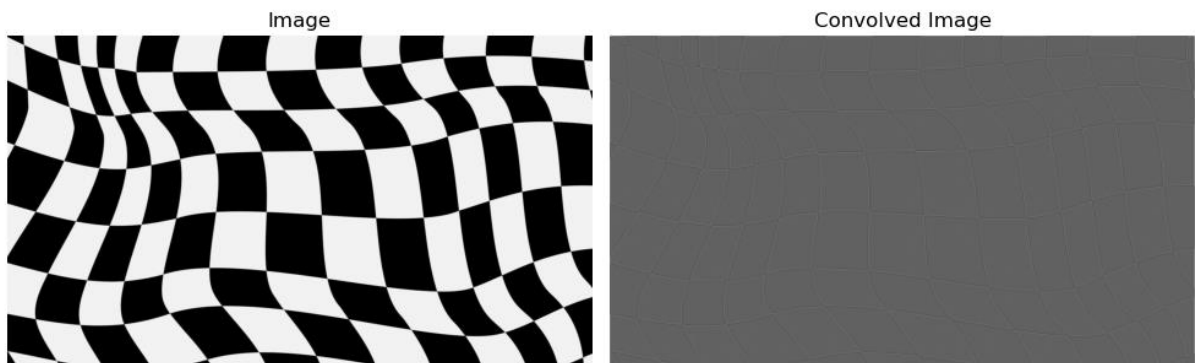
2. What is the role of padding in the convolution operation, and what are the different types of padding?

Padding has two main purposes, to maintain output size and/or to preserve information about the pixels on the edge of the input image. “Valid” padding adds nothing to the edges of the input, and so the output becomes smaller because the kernel cannot be centered over the edge pixels, and for that reason it skips them. Constant, same and zero padding are all names for essentially the same process. In all three, one or more layers of the same value are added to the sides of the image so that as the kernel works its way across the edges, no detail is lost. This results in an output that is the same size as the input. Also, since in 8 bit image displays 0 represents black, edges tend to look darker or may fade to black. Edge (aka. replicate) padding repeats the border pixels, adding one or more layers around the input. Essentially, we stretch the edge colors outwards which prevent the dark borders that happen in zero padding while keeping the output the same size as input. Reflect and symmetric padding both mirror the edge pixels, which create natural looking edges with none of the harsh lines that can appear in zero padding. The difference is that reflect padding includes the edge pixel in the mirror, in effect duplicating it, and symmetric padding does not repeat the edge pixel. By repeating the edge pixel, the edge brightness may be exaggerated. Alternatively, by not duplicating the edge pixel, symmetric padding can sometimes create artificial patterns at the borders if we are applying filters. Finally, wrap padding is where we essentially connect the opposite sides of the image. This can work great on repeating patterns, but on photos it can create strange patterns since you may be mixing unrelated parts of the image.

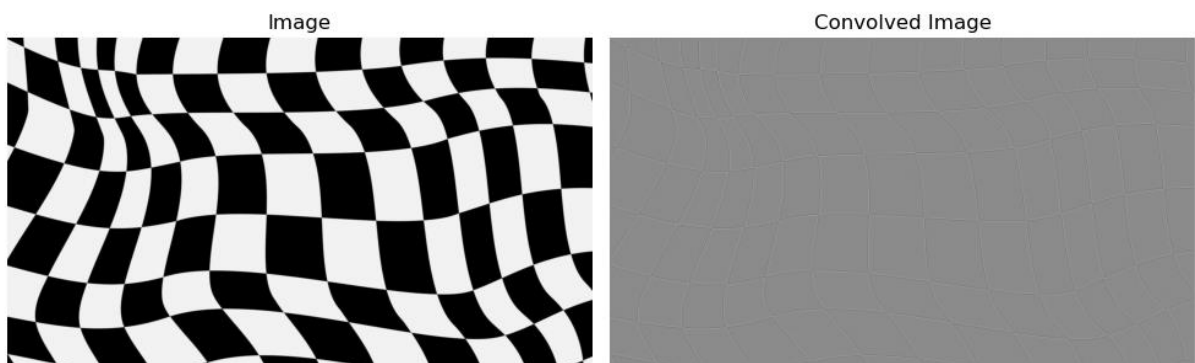
- *Constant Padding (padded with 0)*



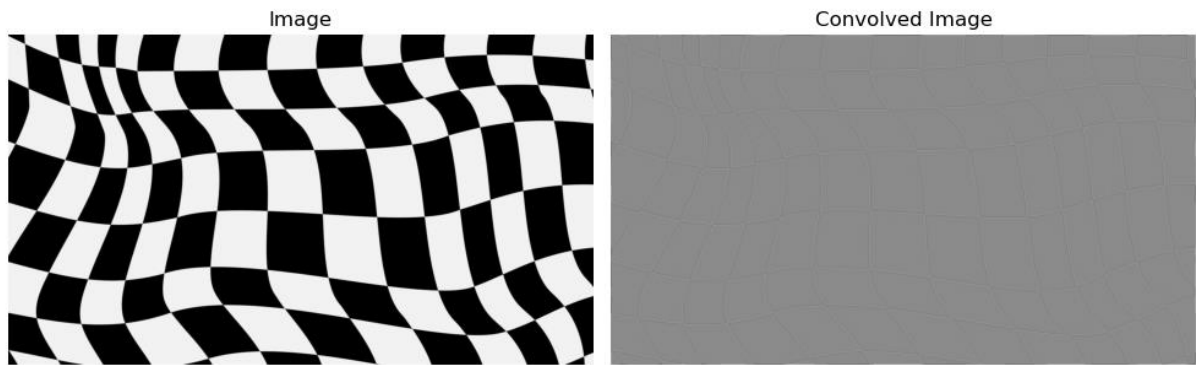
- *Constant Padding (padded with 255)*



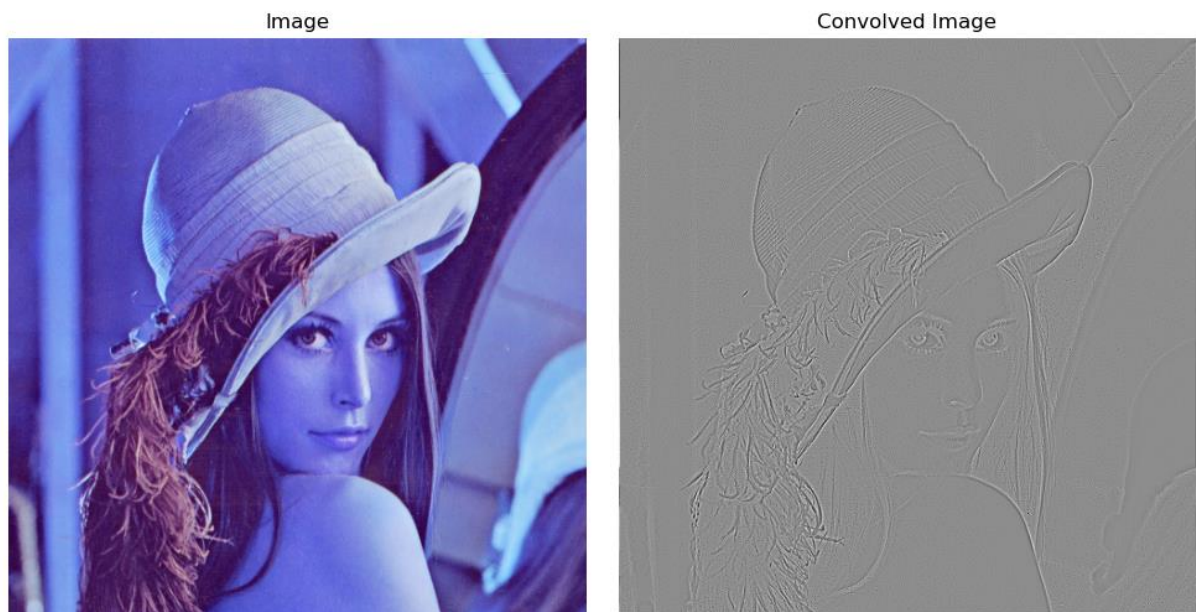
- *Reflect Padding*



- *Wrap Padding (on a pattern)*



- *Wrap Padding (on a picture)*



3. How does the stride parameter influence the convolution operation?

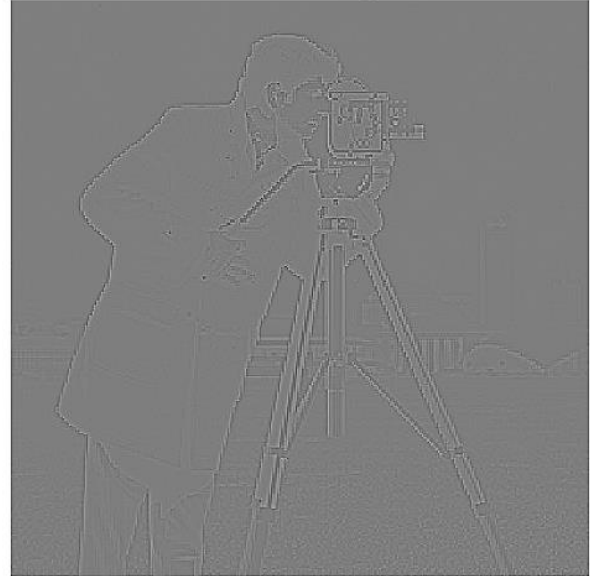
The stride is how big the steps are between image patches. Basically, if your stride is small (i.e. stride = 1) then the kernel moves across the input image one pixel at a time, resulting in a very precise convolution process and a larger output. It follows that with a large stride (i.e. stride = 2+) the kernel moves across the image 2 pixels at a time, and so on. This means that pixels are being skipped, which will result in less detail and a smaller output. This can be helpful when you want to focus on the big picture by accentuating the large features.

- *Stride 1*

Image



Convolved Image

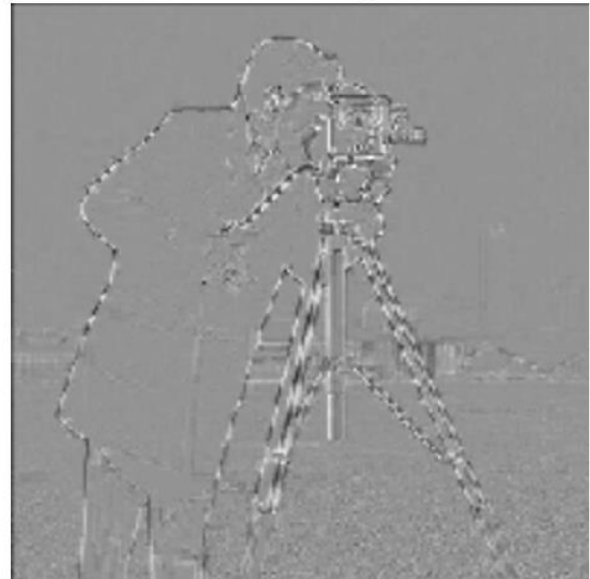


- *Stride 2*

Image



Convolved Image

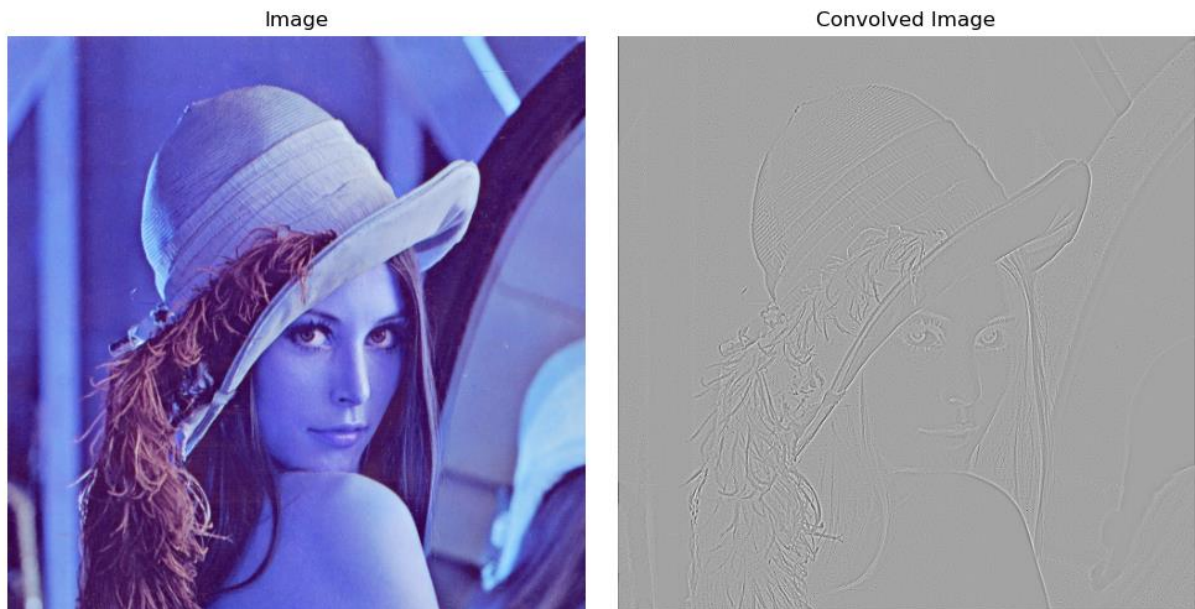


4. How do small kernels (e.g., 3x3) and large kernels (e.g., 5x5 or 7x7) affect the outcome of the convolution operation?

Small 3x3 kernels are better for showing edges and texture. They look at smaller sections, which accentuate the small details and sharp edges. The downside to this type of kernel is that it can leave a grainy appearance and accentuate flaws from camera sensors. Large 5x5 kernels look at a bigger section, which causes edges and details to blend together a bit. This is because 5x5 kernels blend together more pixels when doing the elementwise multiplication. This type of kernel balances noise reduction and blurring effects well so it can be very useful. Large 7x7 kernels look at big sections and this causes the resulting image to be blurrier with smooth edges and loss of detail. This

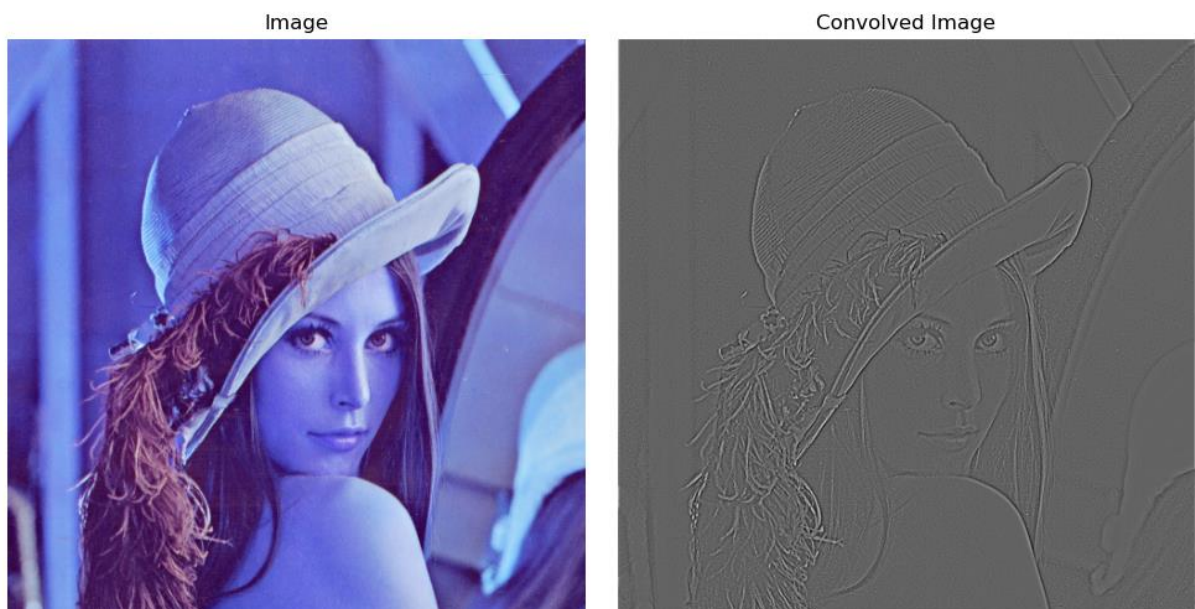
is because the convolution process blends so many pixels together that everything starts to blend together. The upside to this is that it can make images look cleaner and softer, which can be desirable for display but unhelpful for analysis.

- *3x3 kernel*

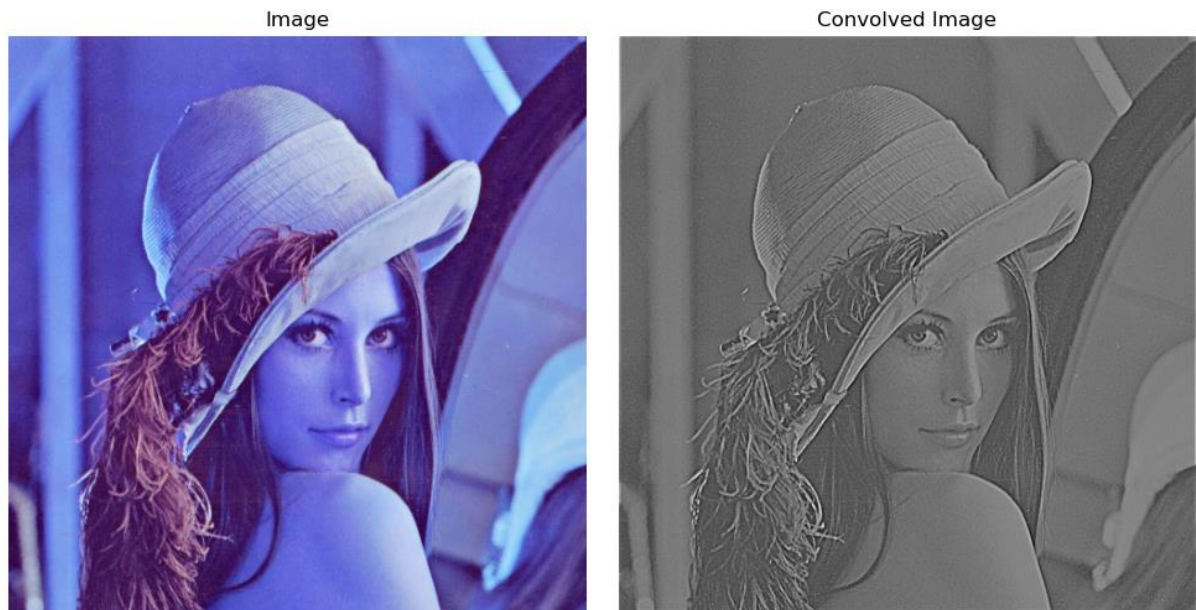


-

- *5x5 kernel*

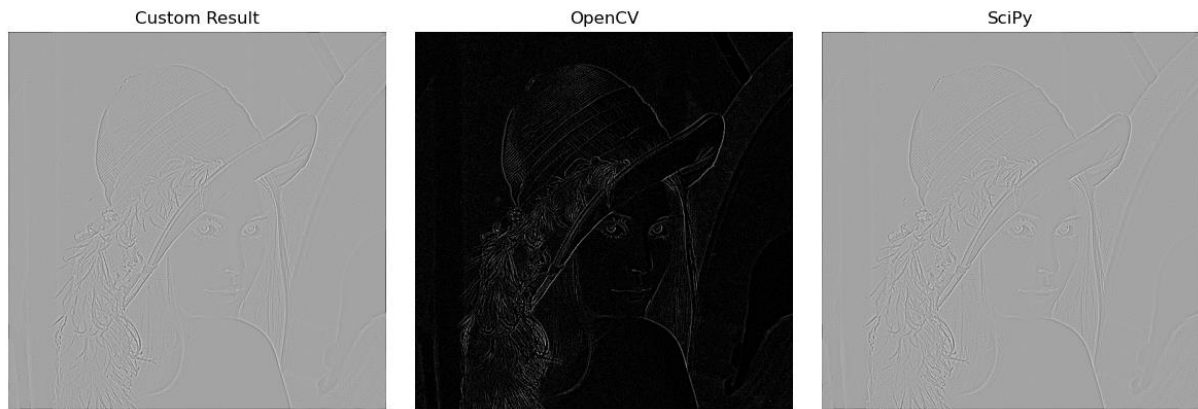


- *7x7 kernel*



5. Why is OpenCV's convolution result so different from your implementation and SciPy's convolution?

There are a few reasons why OpenCV's convolution results in such a different outcome than SciPy or custom convolution programs. First, OpenCV does not flip the kernel and so it is technically a cross-correlation instead of convolution. This works in part because with symmetric kernels, flipping makes virtually no difference. SciPy does flip the kernel, and it follows the formula for pure mathematical convolution. Flipping the kernel is a key step for the mathematical side of convolution, since in convolution you are essentially doing a dot product between the flipped kernel and image segments. Basically, convolution measures how similar the reversed kernel is to the image, while correlation measures how similar the plain kernel is to the image. Basically, one tends to accentuate the light parts of the image while the other will accentuate the dark parts. Last, the border handling methods for our program and SciPy's are different from that which OpenCV uses. Both OpenCV and SciPy copy the edge pixels by mirroring them, but OpenCV doesn't repeat the very last pixel while SciPy does.



LINEAR FILTERING

6. Briefly describe the goal of each linear filter. Use your own words.

Edge detection filters:

Laplacian Filter: This filter checks how the brightness changes around each pixel using a Laplace kernel. It helps find all the edges in the image, no matter the direction.

Sobel Filter: This one uses two separate kernels — one for detecting horizontal edges and one for vertical edges. Then it combines the results to show all the main edges and contrasts in the image.

Blurring filters:

Average Box Filter: It replaces each pixel with the average value of its neighbors, making the image look smoother and less detailed.

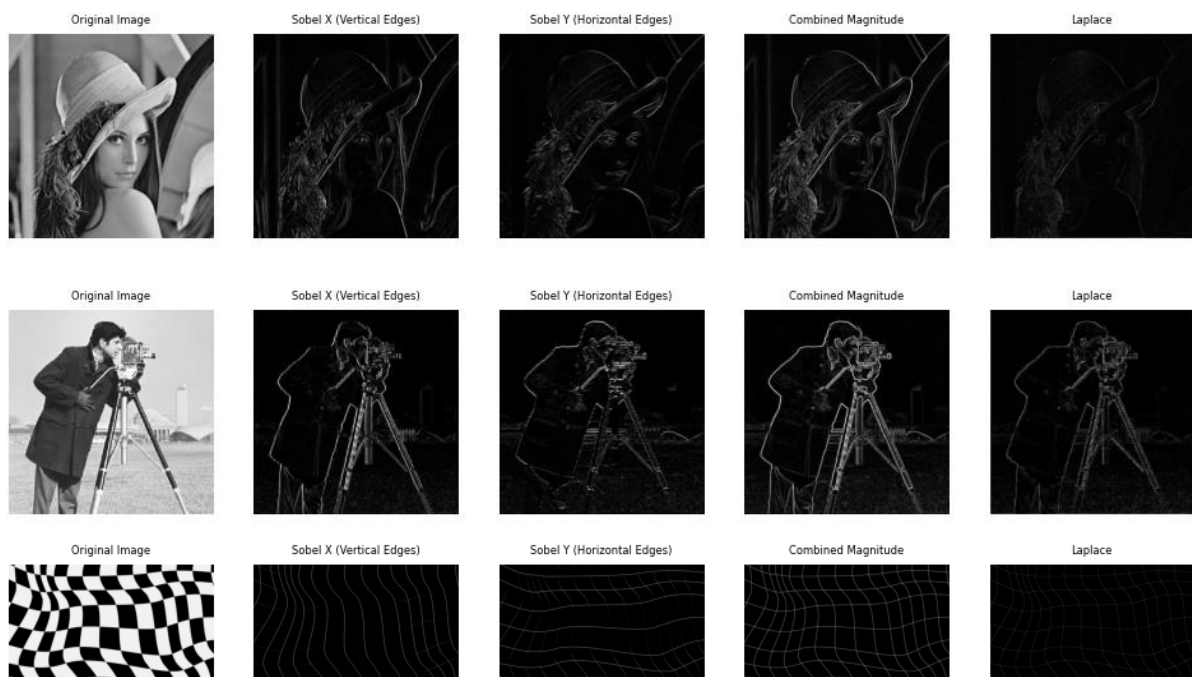
Gaussian Filter: Similar to the average filter, but it gives more importance to the pixels that are closer to the center. This makes the blur look softer and more natural.

7. The Sobel filter detects edges in specific directions (horizontal or vertical). What are the potential advantages and disadvantages of using a directional filter like Sobel versus a non-directional filter like Laplacian in edge detection tasks?

The Sobel filter finds edges in a certain direction, like horizontal or vertical, and it's better at handling noise because it uses the first derivative and smooths the image a bit. It's good for high-contrast pictures and gives you an idea of where the edges are going.

The Laplacian filter looks for edges in all directions at once, but since it uses the second derivative, it's more sensitive to noise. So Sobel is better when you want cleaner and more directional edges, while Laplacian is better if you just want to find all edges in general.

In the examples above, the Sobel filter shows clearer and more directional edges, as seen in the vertical and horizontal results. When combined, it highlights the main contours of the objects very well. The Laplacian filter detects edges in all directions, but the image looks noisier and less defined. Overall, Sobel gives cleaner and more accurate edges, while Laplacian shows more edges but with more noise.

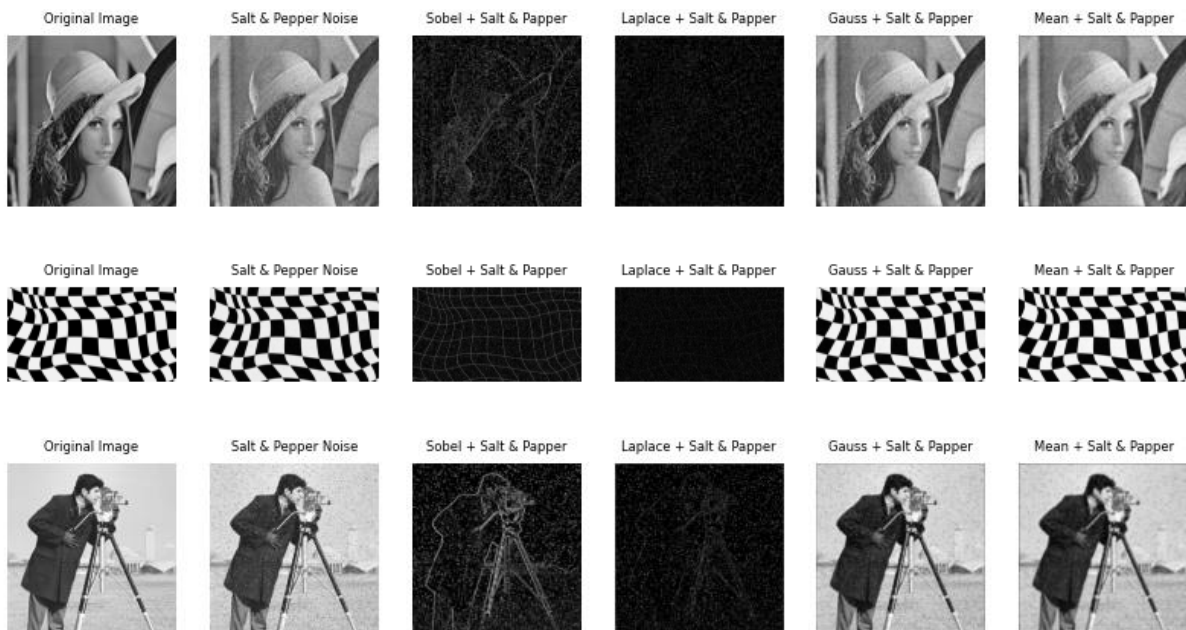


8. After adding salt-and-pepper noise to the image, which filter (Laplacian, Sobel, Mean or Gaussian) is the most effective at reducing noise? Justify your answer with observations from the filtered outputs.

The Mean and Gaussian filters proved to be the most efficient ones at reducing noise because, in the examples above, the images processed with these filters look much smoother and cleaner. Most of the white and black “salt and pepper” dots disappeared, and the main shapes and details of the images are still visible.

In contrast, the Sobel and Laplacian filters didn't remove the noise — they made it more visible by detecting it as edges, so the images look rougher and darker.

In comparison, the Sobel and Laplacian filters don't reduce noise well — they actually make it worse by highlighting the random white and black pixels as if they were edges. That's why the Mean and Gaussian filters are more effective for cleaning the image while keeping a good level of detail.



TEMPLATE MATCHING (OPTIONAL)

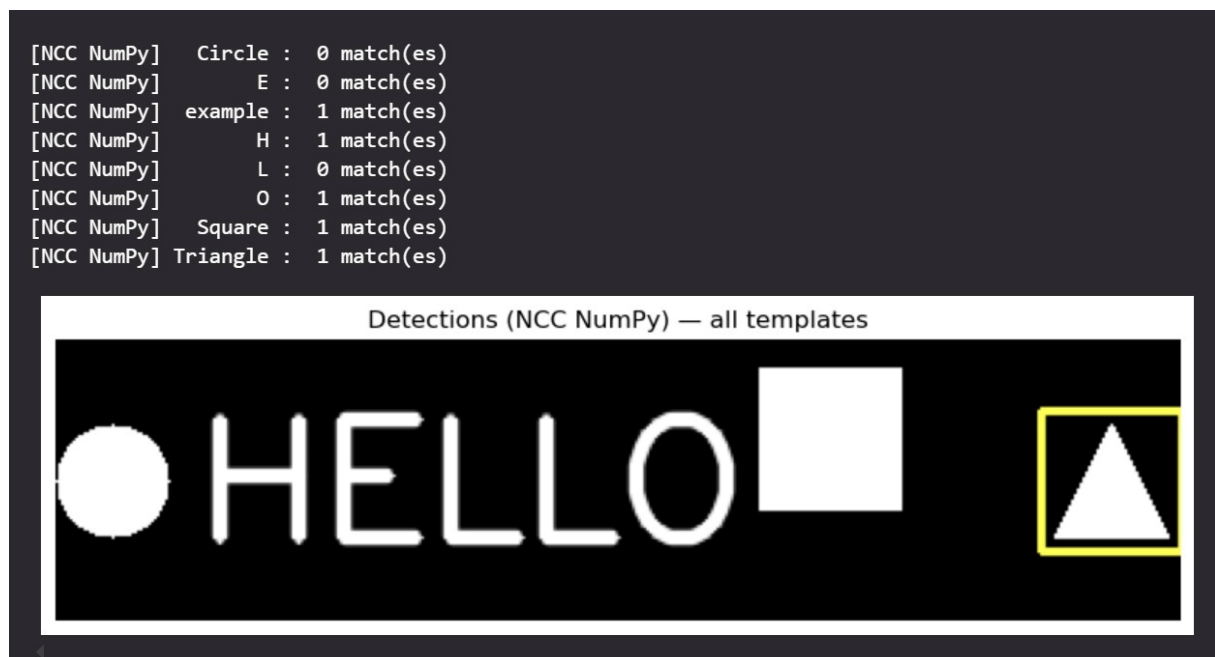
9. What is the goal of template matching in computer vision?

Briefly describe the purpose and typical applications of template matching.

Template matching is a fundamental operation in computer vision whose purpose is to locate occurrences of a smaller image the *template* —within a larger *scene image*. The method systematically slides the template across the scene and compares the similarity between the two image regions. In this way, it can identify where known shapes, letters, or objects appear.

In this project, template matching was used to detect both letters (H, E, L, O) and geometric shapes (circle, square, triangle) from the “Hello Scene” image. The program performed a similarity comparison for each template and drew colored bounding boxes around the locations where matches were found.

Typical applications of template matching include industrial inspection (detecting missing or misaligned components), medical image analysis (locating anatomical structures), and augmented reality (tracking predefined markers). It is also commonly used in document analysis and robotics for simple object recognition tasks.



Template matching applied to the “HELLO” scene using NCC (NumPy implementation). Each template (H, O, triangle, etc.) was successfully detected and highlighted with bounding boxes.

10. Explain the role of Normalized Cross-Correlation (NCC) in template matching. Why is normalization important in comparing image regions and templates?

The most important similarity measure used in this project is the **Normalized Cross-Correlation (NCC)**. NCC quantifies how well the template and a region of the scene image align, while also compensating for brightness and contrast variations. Without normalization, two regions could appear similar just because they are both bright or dark, not because they share the same structure.

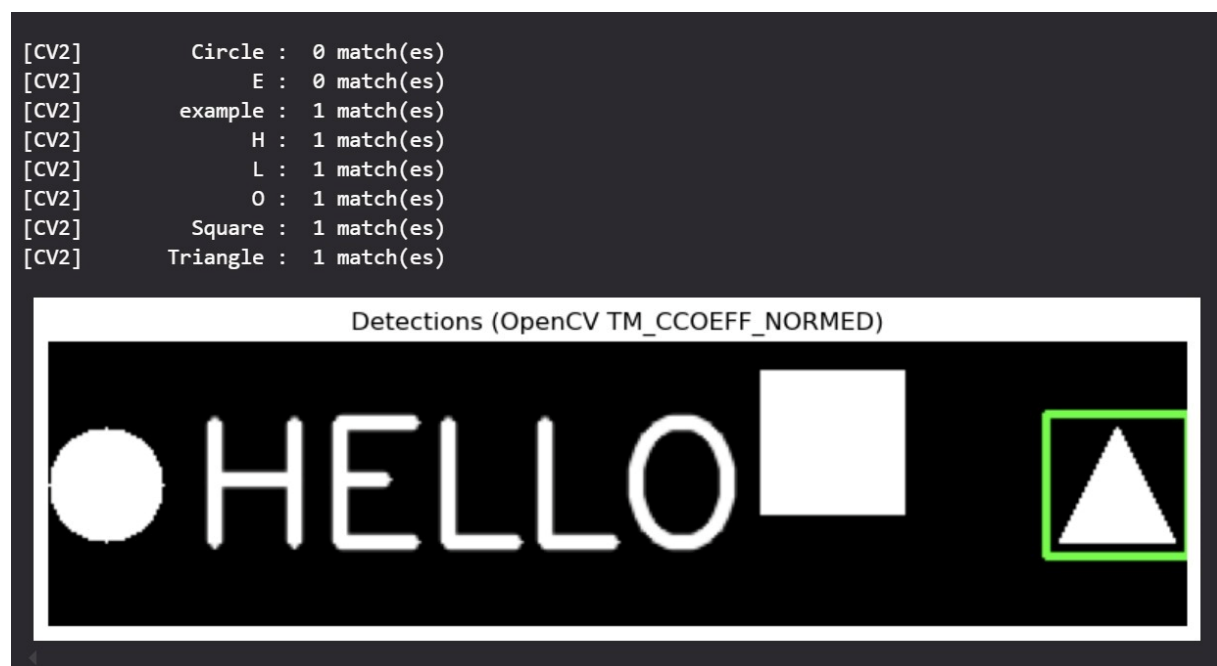
Mathematically, NCC is defined as:

$$NCC = \frac{\sum(I(x,y) - \mu_I)(T(x,y) - \mu_T)}{\sigma_I \cdot \sigma_T}$$

where $I(x,y)$ and $T(x,y)$ are pixel intensities from the scene and template, and μ_I and σ_I represent their mean and standard deviation. This normalization ensures that the resulting correlation values always lie between **-1 and +1**, making them independent of overall image intensity.

In practice, the code computes this using the function `normalized_cross_correlation(scene, tpl)` implemented with NumPy. The output is a 2D *correlation map* where each value represents the similarity between the template and a particular region in the image. Bright regions indicate stronger matches.

Normalization is therefore crucial because it enables reliable matching even when illumination or contrast vary between the template and the scene. The figure below shows how both the manual NCC implementation and OpenCV's built-in correlation method achieve nearly identical results.



Comparison between manual NCC (NumPy) and OpenCV TM_CCOEFF_NORMED results. Both methods produced nearly identical bounding boxes for detected shapes.

11. In your own words, explain the meaning of an NCC value of 1, 0, and -1.

What does each of these values indicate about the similarity between the image region and the template? Relate your answers to your output results.

Each pixel in the NCC result map corresponds to a similarity score between the template and the corresponding region in the scene:

- **NCC = +1** → Perfect match. The region is identical to the template in structure and intensity.
- **NCC = 0** → No correlation. The template and region are unrelated (random similarity).
- **NCC = -1** → Perfect inverse correlation. The region is the opposite of the template (e.g., black vs white).

In the project results, regions with NCC values above a threshold (typically 0.75–0.80) were selected as valid detections. After correlation, the algorithm applied **non-maximum suppression (NMS)** to keep only the strongest bounding boxes and remove overlaps.

This process effectively detected all letters and shapes from the scene. A lower threshold produced more detections, including some false positives, while a higher threshold resulted in cleaner, more reliable matches.

```
[NCC NumPy] Circle : 0 match(es)
[NCC NumPy] E      : 0 match(es)
[NCC NumPy] example : 1 match(es)
[NCC NumPy] H      : 1 match(es)
[NCC NumPy] L      : 0 match(es)
[NCC NumPy] O      : 1 match(es)
[NCC NumPy] Square : 0 match(es)
[NCC NumPy] Triangle : 1 match(es)
```

Detections (NCC NumPy) - all templates



Detected templates using NCC NumPy, showing matches for H, O, and triangle shapes. Brighter regions in the NCC map correspond to higher similarity scores.



OpenCV correlation results. The green boxes mark the same template detections computed with `cv2.matchTemplate()` using `TM_CCOEFF_NORMED`.

Conclusions

Through this experiment, template matching using normalized cross-correlation successfully identified all relevant templates in the “Hello Scene” image. The normalization step ensured consistency across different brightness conditions, while non-maximum suppression improved detection clarity by eliminating redundant matches.

Both the **custom NumPy implementation** and **OpenCV’s optimized version** produced equivalent results, demonstrating the reliability of the NCC metric. However, the method’s main limitation remains its sensitivity to **rotation and scale**, if the template is rotated or resized, the match quality significantly decreases.

Overall, this experiment provided a clear understanding of how template matching operates mathematically and practically. It also highlighted the trade-offs between computational cost, robustness, and simplicity fundamental considerations in real computer vision tasks.